

Roll No: 2024114010

04 April 2026

CS7.401 Introduction to NLP Quiz 2 (Set 1) (S26)

IIIT Hyderabad

General Instructions

- a Clearly write your roll number on question paper.
- b The quiz consists of two parts:
 - **Part A (MSQ — Multiple Select Questions) has 5 questions.**
 - **Part B has 3 Subjective questions.**
- c In case of MSQ ensure that your responses are clearly marked. A simple circle around the chosen option (A, B, etc.) or a tick mark is sufficient. Any ambiguous or confusing markings will be penalized (-5).
- d The total marks for the quiz will be calculated as:
Total Marks = Part A + Part B
- e Part A instructions:
 - (a) These are 5 Multiple Select Questions (MSQs). Each question may have **one or more correct options**.
 - (b) Each question will be evaluated based on the following rules (in order):
 - You will get **1 mark per correct option** selected.
 - Unattempted questions will be not evaluated.
 - If there is even one incorrect option selected, all correct options selected will get **0 marks** and each incorrect option selected will get **-1 mark**.

Example Calculation:

Question has 4 correct (A, B, C, D) and 1 wrong(E) option.

Student selects A, B, D and E.

Score: $3 * 0 (= 0) + (1 * -1) = -1 \text{ marks}$.

Part A

Question 1

Self-Attention layer is computationally more efficient than a recurrent layer.
Under what conditions and reasons is this true?

c, d, e

- a Self-Attention is $O(n * d)$, while Recurrent layers are $O(n * d)$.
- b Self-Attention reduces computation because it eliminates the need for matrix multiplications present in recurrent layers.
- ✓ In typical NLP tasks where d (e.g., 512) is much larger than n (sequence length), $n^2 * d$ is often less than $n * d^2$.
- ① X ✓ Self-Attention is more efficient because it processes tokens independently without any interaction between them.
- ✓ When the sequence length n is smaller than the representation dimensionality d .

Question 2

Why was sinusoidal approach chosen for positional encoding instead of learning fixed embedding for each position?

- a Fixed embeddings are mathematically impossible to use in a non-recurrent architecture.
- b Learned embeddings were found to result in massive over-fitting on short sentences.
- c The choice was purely aesthetic and had no impact on the final performance or BLEU scores.
- ① d Sinusoidal encodings guarantee better performance than learned embeddings on all NLP tasks.
- ✓ Functional encodings allow the model to extrapolate to sequence lengths longer than those encountered during training.

Question 3

What are the intuitive and structural benefits of using h parallel heads instead of one large attention function?

- ③ ✓ It allows the model to jointly attend to information from different representation subspaces at different positions.
- ✓ It allows for parallelization across different perspectives of the same input data.
- c It significantly reduces the total number of trainable parameters in the model.
- ✓ By using smaller dimensions for each head ($d_k = d_{model} / h$), the total computational cost remains similar to single-head attention.
- e Each head specializes in a specific linguistic function (e.g., one for verbs, one for nouns).

Question 4

Which of the following are true with respect to the Q , K , V and attention in a Transformer?

- ④ ✓ In Self-Attention, Q , K , and V all come from the previous layer's output.
- ✓ In Encoder-Decoder Attention, the Q come from the previous decoder layer, while the K and V come from the encoder's output.

- c Attention weights are computed using only the K and V , without involving Q .
- d ✓ Encoder-Decoder Attention allows the decoder to focus on specific parts of the input sequence.
- e ✓ In Self-Attention, Q and K must have the same dimension d_k , but V can have a different dimension d_v .

Question 5

Why is a specific combination of Residual Connections and Layer Normalization ($LayerNorm(x + Sublayer(x))$) used in Transformer?

- a Layer Normalization is significantly faster to compute on a GPU than Batch Normalization.
- b ✓ LayerNorm normalizes across the features of a single example, making it independent of other examples in the batch.
- c The combination is necessary to reduce the quadratic time complexity of self-attention.
- d ✓ The residual connection ensures that the signal from lower layers can pass through to higher layers unimpeded.
- e LayerNorm is only applied to the Queries and Keys, never to the Values or Feed-Forward outputs.

Part B

10 Marks

Question 1

4 Marks

Let's say a Transformer 'A' uses Multi-Head Attention with $h = 8$ heads. The model dimension is $d_{\text{model}} = 512$, meaning for each head $d_k = d_v = 64$. A transformer 'B' is trained using a single attention head where $d_k = d_v = 512$; this is mathematically equivalent to the 8-head setup as both ultimately perform linear transformations resulting in a single 512-dimensional output vector. Then why do we train on more heads? Also why do you think it would yield better contextual representations?

Every head in multi-headed attention focuses on only one part of context provided. By training on a larger number of heads, we are ensuring that the model pays attention to all forms of linguistic features, and as a result, some heads might learn the syntactic knowledge of the sentence, another might learn about the theta roles in the sentence. This is very handy when it comes to tasks like coreference/anaphora resolution, as unlike a single headed attention, an MHA model would have acquired & encoded the information from the multiple candidates for the coreferent. Similarly, in case of Word Sense Disambiguation, MHA outperforms SHA as the model is capable of analysing and learning more of the surrounding context to the word. Hence we train on multiple heads, as it provides better contextual representations, as a result of having each head focus on a different part of the sentence, acquiring a more complete. And since the reduced values of d_k & d_v ensure that the model is mathematically the same as the SHA model, training the MHA⁴ takes no more complex than training the SHA.

Question 2

2 Marks

A purely self-attention-based network operating without any positional encodings is formally described as permutation equivariant. What does this mean and why does it happen? Give a brief description of how positional encodings were implemented in the original Transformer architecture paper.

Permutation invariance states that self-attention treats language as a Bag of Words, and does not discriminate b/w different word orderings.

For example, the sentence "Zuko made his uncle tea" and "His uncle made Zuko tea" would have the same attention provided to each of the words regardless of where they appear. This results in the model not attending to certain semantic features like theta roles and syntactic features like Subject, verb, and object. This is caused due to SA dealing solely with the word embeddings (generated through Word2Vec or GloVe) which do not encode anything beyond the semantic representation of the word.

In the original paper dealing with Transformers, positional encoding was implemented using a sinusoidal fn $[\sin(\lambda/10000)^i]$ where i was the position of the word. This was then concatenated with the word embedding to create a positional word embedding.

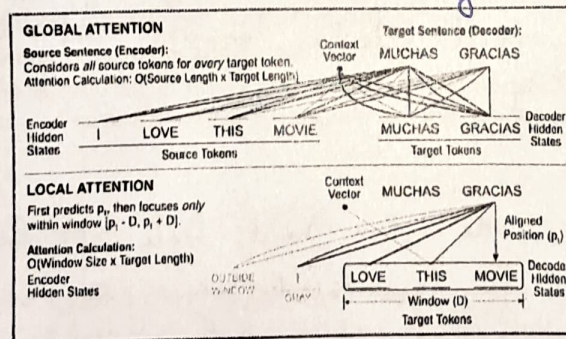


Figure 1: Global Vs Local Attention

Question 3

4 Marks

You are already familiar with Global Attention with LSTM, where the model considers every hidden state of the encoder for every step of the decoder. However, another approach is Local Attention. Local Attention (Figure 1) acts as a hybrid between hard and soft attention. Instead of looking at the

entire source sentence, it predicts an aligned position (p_i) and only focuses on a small, fixed-size window of tokens around that position. This makes it computationally more efficient and less likely to blur context in very long sentences. When building an LSTM-based Neural Machine Translation (NMT) system, which attention mechanism (Global or Local) is intuitively more effective for the following translation directions? Justify your choice based on the structural differences between English and Indian languages.

a English $\rightarrow$ Indian Language (e.g., English to Hindi/Tamil)

b Indian Language $\rightarrow$ English (e.g., Marathi/Bengali to English)

① Local ~~attention~~ feels intuitively better for an NMT system translating from English to any Indian language. This can be reasoned as follows *will create issue from Data PK*

(i) English has a fixed word order, while Indic languages have free word ordering, allowing multiple correct possible translations to exist because of case marking. By choosing to provide local attention to specific windows of tokens, we can ensure that the translation is done correctly.

(ii) English allows extensive relative clausuring, which is usually encoded adjectivally in Indic languages or by using markers like ~~which~~ *जिसकी - उसकी*. Local attention ensures that this parsing and conversion is done accurately.

② Global attention, on the other hand, feels apt for NMT systems performing translation from Indic languages to English. Since Indic languages are generally free word ordered, it might be unlikely to find the position of alignment (p_i). Also, certain (Dravidian) Indic languages encode semantic information by performing morphological processes on the verbs. All of this requires the model to pay attention to the whole sentence to truly understand the context and provide an accurate translation.

(How does Local Attention work with tokenisers like BPE or word-piece?)